Lucas Gómez · Martín Bentura
Montevideo, noviembre de 2026

Valheim Server Manager en AWS

Introducción a AWS Cloud · Proyecto final · Servidores de Valheim que se prenden desde un panel y se apagan solos

Armamos en AWS un sistema para manejar servidores de Valheim sin tener una máquina prendida todo el día: desde un panel web se crean, se prenden y se apagan los servidores, y si no queda nadie jugando se apagan solos. El mundo se guarda en S3 entre partida y partida. Toda la infraestructura está en cuatro templates de CloudFormation y el ciclo completo (crear, prender, jugar, backup, apagar) lo probamos de punta a punta en el Learner Lab. Cuesta unos 0,09 USD por hora de juego, contra unos 65 USD por mes que saldría dejar la misma instancia prendida 24 horas.

1. QUÉ HACE LA APLICACIÓN

Un servidor de juego tiene estado, corre en una sola instancia por mundo y habla UDP, así que no se puede balancear ni repartir entre máquinas. Lo que sí nos da la nube es escalar a cero: pagar cómputo solo mientras hay gente conectada. Por eso separamos la parte que administra (serverless: S3, API Gateway, Lambda, DynamoDB, EventBridge) de la parte que corre el juego (EC2 dentro de un Auto Scaling Group, detrás de un balanceador UDP).

QUÉ HACE	CON QUÉ LO RESOLVEMOS EN AWS
Ver los servidores y su estado (apagado, arrancando, en línea, jugadores)	Panel estático en S3 que llama a una HTTP API; la Lambda `api-handler` lee DynamoDB y los Auto Scaling Groups
Crear y eliminar servidores desde el panel	La Lambda `provisioner` crea o borra el stack de CloudFormation del servidor usando `04-servidor.yaml` guardado en S3
Prender y apagar con un botón	La Lambda cambia la capacidad deseada del ASG del servidor entre 0 y 1
Dirección fija para los jugadores	Network Load Balancer con listeners UDP 2456 y 2457; el ASG registra la instancia sola
El mundo no se pierde	S3 con versionado: se restaura al arrancar, backup cada 20 minutos y al apagar
Se apaga solo si no hay nadie	Métrica `PlayersOnline` por minuto, alarma de CloudWatch a los 30 minutos en cero, EventBridge llama a `idle-stop`

QUÉ HACE	CON QUÉ LO RESOLVEMOS EN AWS
Apagado sin perder progreso	Lifecycle hook del ASG + Lambda `graceful-shutdown` + Run Command: para el juego (que guarda), sube a S3 y libera el hook
Historial y avisos	Eventos del ASG, S3 y CloudFormation llegan por EventBridge a las Lambdas, quedan en DynamoDB y se ven en el panel; webhook de Discord
Red de seguridad de costos	Apagado programado a las 04:00, límites de 10 servidores creados y 8 prendidos, tags de costo

2. ARQUITECTURA

El plano de control vive fuera de la VPC: S3 sirve el panel, API Gateway recibe las acciones con una clave en el header, seis Lambdas en Python hacen la lógica, DynamoDB guarda estado e historial, EventBridge mueve los eventos y SQS recibe lo que falla. El plano de datos es una VPC propia en dos zonas de disponibilidad, con un NLB UDP y un Auto Scaling Group de mínimo 0 y máximo 1 por servidor; la instancia corre Valheim en Docker y guarda el mundo en S3.

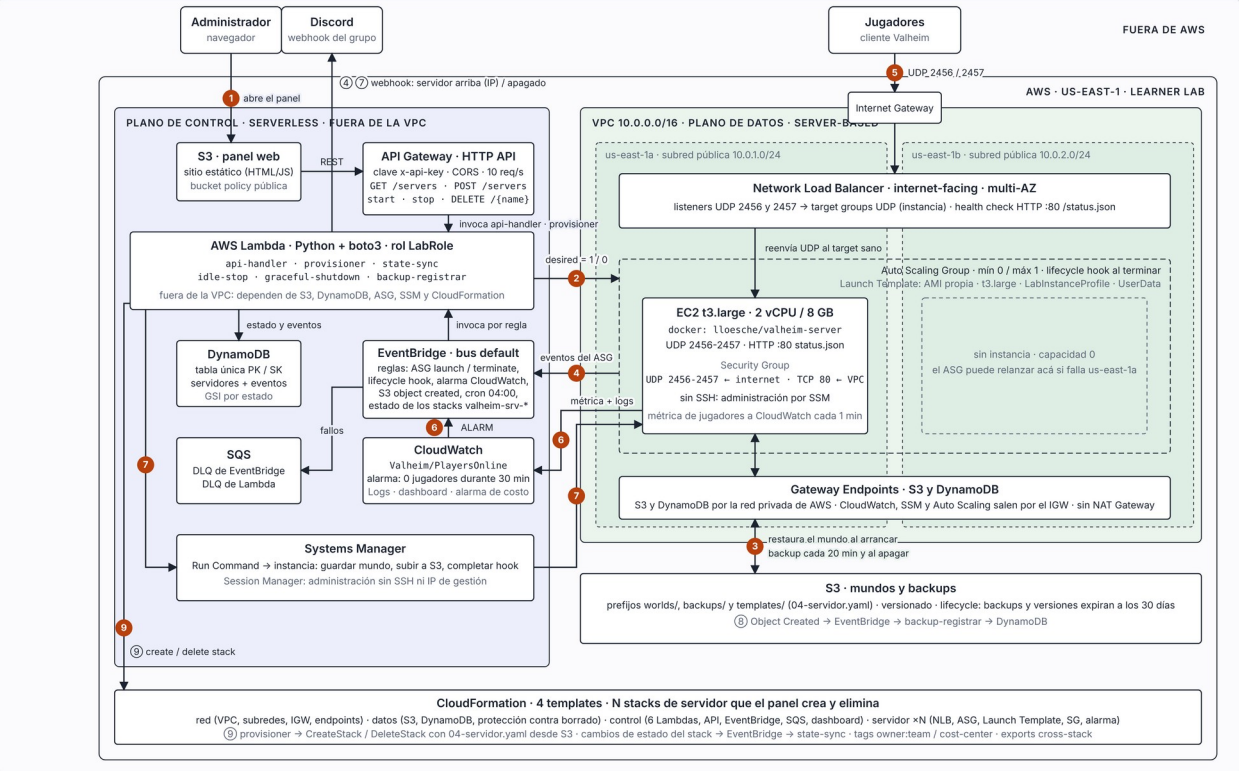


Figura 1. Componentes y flujo. A la izquierda el plano de control fuera de la VPC; a la derecha la VPC con el servidor dentro de un ASG de 0 a 1 detrás del NLB. Los números marcan el flujo principal y el 9 la creación de servidores desde el panel.

El flujo

- Prender: el panel llama a la API con la clave `x-api-key`; `api-handler` pone `DesiredCapacity = 1` en el ASG del servidor. Si el anterior todavía se está apagando responde 409, para no restaurar un mundo viejo.
- Arranque: la instancia baja el mundo desde S3 por el Gateway Endpoint, levanta el contenedor y el ASG la registra en el NLB. El evento de lanzamiento llega por EventBridge a `state-sync`, que marca el estado y avisa en Discord; cuando el juego responde, la propia instancia marca En línea.
- Apagado: cada minuto la instancia publica `PlayersOnline`; con 30 minutos en cero la alarma dispara `idle-stop`, que pone la capacidad en 0. El lifecycle hook frena la terminación hasta que `graceful-shutdown`, por Run Command, para el juego, sube el mundo a S3 y libera el hook. La misma Lambda corre por cron a las 04:00 por las dudas.
- Crear y eliminar: Nuevo servidor llama a `POST /servers`; `provisioner` valida nombre, mundo, contraseña y tamaño, controla el límite y le pide a CloudFormation el stack `valheim-srv-<nombre>`. Los cambios de estado del stack llegan por EventBridge a `state-sync`, que deja el servidor en Apagado cuando termina. Eliminar borra el stack solo si el servidor está apagado; el mundo queda en S3.



Figura 2. Ciclo de vida de un servidor. CREATING y DELETING son estados del stack de CloudFormation; los otros cuatro los escriben las Lambdas con los eventos del ASG. La capacidad deseada del ASG es el único interruptor.

Decisiones que tomamos

ELEGIMOS	EN VEZ DE	POR QUÉ
ASG de 0 a 1 con el mundo en S3	Una EC2 que se detiene y arranca con su EBS	La instancia es descartable: cuando nadie juega no hay nada que pagar, y si falla la instancia o la AZ el ASG la relanza. A cambio hay que guardar el mundo a propósito (backup, lifecycle hook, versionado).
Network Load Balancer	Elastic IP en la instancia	Valheim es UDP y con estado, un ALB no sirve. El NLB da DNS fijo, health checks y registro automático. Es lo más caro del diseño (unos 16 USD por mes); una EIP haría lo mínimo por 3,65.

ELEGIMOS	EN VEZ DE	POR QUÉ
DynamoDB	RDS	Pocos ítems y acceso por clave. RDS obligaba a meter las Lambdas en la VPC y pagar NAT o Interface Endpoints sin ganar nada.
Lambdas fuera de la VPC y Gateway Endpoints	Lambdas en subred privada, NAT Gateway	Ninguna Lambda necesita recursos privados. Los endpoints a S3 y DynamoDB no tienen costo por hora y el tráfico pesado (mundos y backups) no sale a internet.
EC2 con Docker	ECS o Fargate	Un solo contenedor con estado y 8 GB por servidor. Fargate cuesta cerca del doble por hora y no resuelve el estado; ECS sobre EC2 agrega una capa para orquestar un contenedor por máquina.
CloudFormation como API para crear servidores	Crear los recursos uno a uno con boto3	El mismo template sirve para la consola y para el botón del panel; si algo falla CloudFormation deshace todo solo y borrar es una sola llamada.

3. SEGURIDAD

Lo único que entra desde internet a algo nuestro es el UDP del juego, y entra por el NLB. El panel y la API terminan en servicios de AWS con TLS, y el servidor no tiene SSH ni IP de gestión.

Redes públicas y privadas. La VPC 10.0.0.0/16 tiene en cada AZ una subred pública (10.0.1.0/24 y 10.0.2.0/24) y una privada (10.0.11.0/24 y 10.0.12.0/24). En las públicas van solo los nodos del NLB y el servidor. Las privadas no tienen salida a internet y hoy están reservadas: ahí iría una base relacional si hiciera falta. El servidor no puede ir a una privada sin un NAT Gateway (33 USD por mes), porque necesita salir a Steam; esa es la decisión de costo que lo deja en la pública.

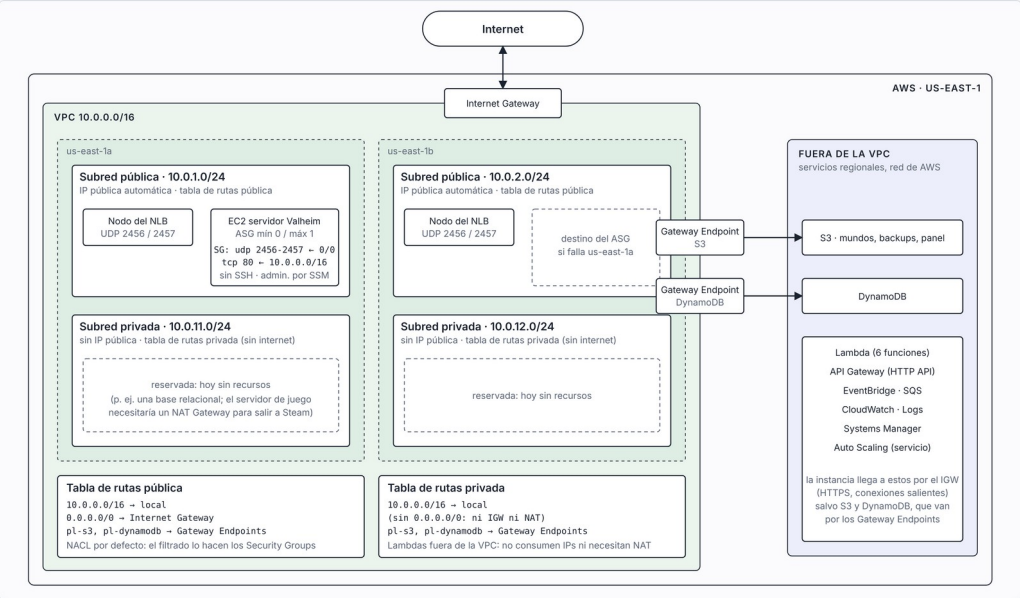


Figura 3. Redes: dos AZ con una subred pública y una privada cada una, Internet Gateway, tablas de rutas, Gateway Endpoints a S3 y DynamoDB y los servicios regionales fuera de la VPC.

Security groups. El servidor acepta UDP 2456-2457 desde cualquier origen y TCP 80 solo desde la VPC (health check del NLB). Nada más entra; el resto son conexiones que la instancia inicia. El NLB solo tiene listeners UDP y la API tiene throttling de 10 solicitudes por segundo.

Protocolos seguros. HTTPS en todo el plano de control: panel, API Gateway, SDK de las Lambdas y de la instancia, agente de Systems Manager. La administración es por Session Manager y Run Command, sin puertos abiertos. El juego usa el protocolo UDP de Valheim con contraseña; es lo único que no controlamos.

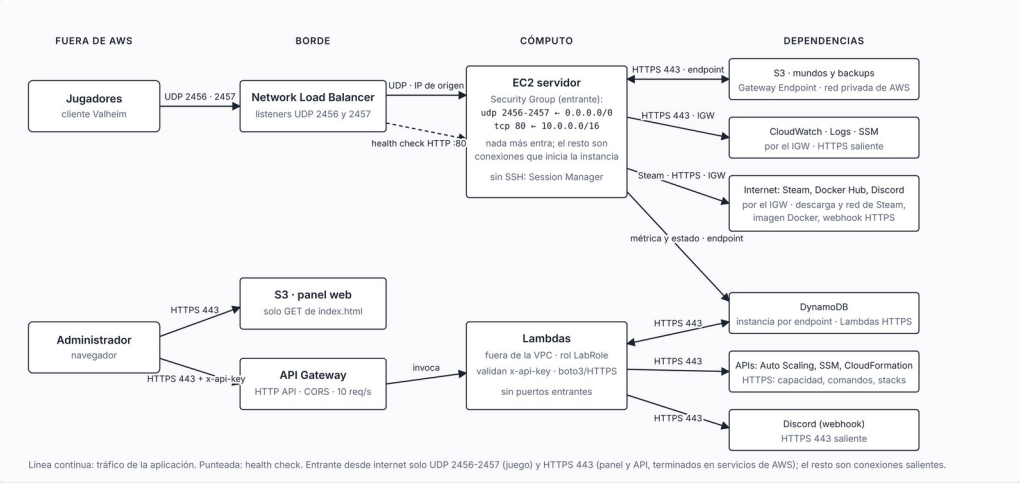


Figura 4. Tráfico: protocolos, puertos y por dónde sale cada conexión.

Encriptación e identidad. EBS cifrado, S3 con SSE-S3, versionado y Block Public Access en el bucket de mundos, DynamoDB cifrada. TLS en todas las llamadas a AWS; S3 y DynamoDB desde la instancia van por los endpoints sin salir a internet. La contraseña del servidor, la clave de la

API y el webhook son parámetros `NoEcho`, nunca están en el código. Las Lambdas usan `LabRole` y la instancia `LabInstanceProfile`: el lab no deja crear roles con permisos mínimos y lo declaramos como limitación.

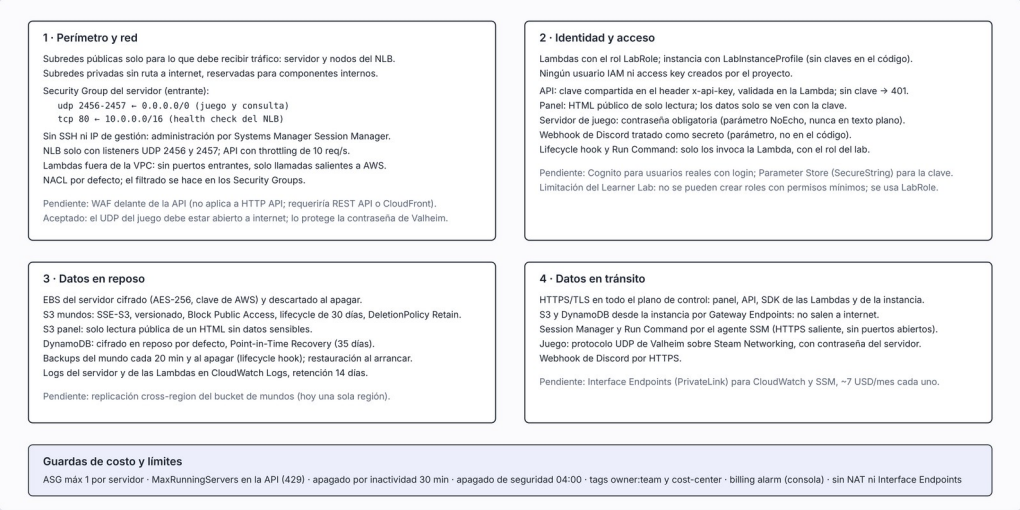


Figura 5. Seguridad por capas: lo que está hecho y lo que queda pendiente o aceptado como riesgo.

4. RESILIENCIA

El sistema aguanta perder la instancia o una AZ entera sin que nadie haga nada: el ASG relanza en la otra AZ y el mundo vuelve de S3, con una pérdida máxima de 20 minutos de juego (el intervalo de backup) y un corte de 2 a 4 minutos. No aguanta perder la región, y eso lo decimos.

Si cae una AZ. El ASG abarca us-east-1a y us-east-1b y el NLB tiene un nodo en cada una. El health check marca la instancia como no sana, el ASG lanza otra en la otra AZ, el UserData restaura el mundo y la instancia se registra sola en el NLB. Los jugadores vuelven a entrar a la misma dirección. S3, DynamoDB, Lambda, API Gateway y EventBridge ya son regionales. **Si cae la región** no hay servicio: la mitigación sería replicar el bucket de mundos a otra región y desplegar ahí los mismos templates, que no lo hicimos por costo y alcance.

La base de datos. DynamoDB guarda cada ítem en tres AZ y tiene Point-in-Time Recovery (35 días). Igual, el dato que importa no es la tabla sino el mundo, que está en S3 con versionado (las versiones viejas duran 30 días) más un zip de backup cada 20 minutos. El stack de datos tiene protección contra borrado y el bucket `DeletionPolicy: Retain`.

Los usuarios. Pierden la conexión durante un reemplazo y vuelven a entrar con la misma dirección; lo construido desde el último backup se pierde (máximo 20 minutos, cero si el apagado fue limpio). El panel es estático en S3 y no depende de la instancia; el estado se arma a partir de los eventos, así que un reinicio no lo deja inconsistente.

Los logs. La consola de Valheim va a CloudWatch Logs con el driver `awslogs`, un grupo por servidor y 14 días de retención; sobrevive a la terminación de la instancia. Las Lambdas

escriben en grupos propios. Los eventos (prendido, apagado, backup, errores) quedan en DynamoDB y lo que falla va a dos DLQ en SQS.

FALLA	QUÉ PASA	RECUPERACIÓN	PÉRDIDA MÁXIMA
Muere la instancia o cae una AZ	El ASG lanza otra, en la otra AZ si hace falta	2 a 4 min	20 min de juego
Falla una Lambda	Reintentos de EventBridge y mensaje a la DLQ	Al corregir	Ninguna
Falla la creación de un stack	CloudFormation deshace los recursos; el panel muestra el motivo	Volver a crear	Ninguna
Se rompe la tabla o se borra un mundo	PITR a un segundo anterior; versión anterior o zip en S3	Minutos	20 min de juego
Cae la región	Sin servicio	No cubierto	Hasta restaurar en otra región

5. ESCALABILIDAD

La unidad de escala es el servidor completo, porque un mundo no se reparte entre instancias. Escalamos en tres direcciones: en el tiempo (de 0 a 1 por servidor, que es lo que ahorra plata), en cantidad (un stack por servidor con el mismo template, creado desde el panel) y en tamaño (tipo de instancia por parámetro). El plano de control escala solo.

DIMENSIÓN	CUÁNDO	CÓMO	LÍMITE
Temporal (0 a 1)	Alguien toca Prender; 30 min sin jugadores; cron de las 04:00	Capacidad deseada del ASG en 1 o 0	`MaxSize: 1` en el ASG
Horizontal (más servidores)	Un mundo llega a los 10 jugadores o el grupo quiere otro mundo	Nuevo servidor en el panel: `provisioner` crea otro stack	`MaxServers` 10 creados y `MaxRunningServers` 8 prendidos; la API responde 429
Vertical (más grande)	CPU alta o falta de memoria con muchos jugadores	Parámetro `InstanceType` y Update del stack; aplica en el próximo arranque	Hasta `large` en el lab

DIMENSIÓN	CUÁNDO	CÓMO	LÍMITE
Plano de control	Siempre, sin configurar nada	Lambda por invocación, DynamoDB on-demand, API Gateway y NLB elásticos	Throttling de la API en 10 req/s

El ASG no lo usamos para repartir carga sino como interruptor con autorreparación: mínimo 0, máximo 1 y la capacidad deseada la manejan las Lambdas. Nos da relanzamiento si falla, registro automático en el NLB, el lifecycle hook para el apagado limpio y los eventos que alimentan el estado. En vez de escalar por CPU, la señal es de negocio: jugadores conectados.

Capacidad configurada: 10 servidores creados y 8 prendidos a la vez, con 10 jugadores por servidor (límite del juego), o sea 80 jugadores. Los techos los pone el entorno: el Learner Lab permite 9 instancias a la vez de hasta tamaño `large`, y AWS da 50 NLB por región. Fuera del lab se piden cuotas y la arquitectura no cambia. Sin límites, un error en el panel podía dejar varios t3.large prendidos y fundir el crédito en días: por eso están en el código, junto con el apagado por inactividad y el de las 04:00.

ECS/EC2 y TCO. Con Fargate un servidor equivalente (2 vCPU, 8 GB) cuesta cerca del doble por hora que un t3.large y sigue sin resolver el estado (habría que montar EFS). ECS sobre EC2 agrega su plano de control para orquestar exactamente un contenedor por instancia. El proyecto completo salió unos 5 a 6 USD en dos semanas; operar con 3 horas de juego por día serían unos 25 USD por mes con NLB o 13 con Elastic IP, contra 65 de un t3.large prendido 24 horas.

6. COSTOS

El costo depende casi solo de las horas que el servidor está prendido: unos 0,09 USD por hora de juego, más 0,56 USD por día de NLB mientras exista el stack del servidor. El plano de control cuesta centavos. Precios on-demand de us-east-1 al 31/10/2026.

COMPONENTE	PRECIO	CUÁNDO SE PAGA
EC2 t3.large (2 vCPU, 8 GB) + EBS gp3 20 GB + IP pública	0,09 USD/h	Solo con el servidor prendido
Network Load Balancer	0,0225 USD/h (16,4 USD/mes)	Mientras exista el stack del servidor
Snapshot de la AMI propia (~5 GB)	~0,25 USD/mes	Siempre
S3 (panel, mundos, backups, ~2 GB)	~0,05 USD/mes	Siempre

COMPONENTE	PRECIO	CUÁNDO SE PAGA
CloudWatch (métrica, alarmas, logs, dashboard)	0 a 1 USD/mes	Siempre
Lambda, DynamoDB, API Gateway, EventBridge, SQS, SSM	~0 USD	Volúmenes mínimos, dentro del free tier

ESCENARIO	COSTO
Todo desplegado con el servidor apagado	~0,6 USD/día (casi todo el NLB)
Proyecto completo, dos semanas, 30 a 40 h de servidor	5 a 6 USD de los 25 asignados
Un mes con 3 h de juego por día	~25 USD con NLB; ~13 USD con Elastic IP
El mismo t3.large prendido 24/7	~65 USD/mes
Hosting comercial de Valheim, de referencia	10 a 15 USD/mes por 10 jugadores

Con 3 horas diarias la solución empata o pierde contra un hosting comercial por culpa del NLB; gana cuando el uso es esporádico, cuando se quieren varios mundos y cuando importa el control. Lo que evitamos por costo: NAT Gateway (33 USD/mes), Interface Endpoints (7 USD/mes cada uno), RDS (15 USD/mes la más chica). Todo lleva los tags `owner:team`, `owner:environment` y `cost-center` para verlo en Cost Explorer, y hay una billing alarm hecha por consola.

7. CÓMO LO DESPLEGAMOS

Cuatro templates de CloudFormation que se crean en orden, porque cada uno exporta valores que importa el siguiente. Los únicos pasos a mano son subir el `index.html` al bucket del panel y el `04-servidor.yaml` a la carpeta `templates/` del bucket de mundos. Lo desplegamos de cero dos veces y las dos veces los cuatro stacks quedaron en `CREATE_COMPLETE` al primer intento, en menos de 15 minutos.

#	STACK	QUÉ CREA	TIEMPO
1	`valheim-01-red`	VPC, 2 subredes públicas y 2 privadas, IGW, tablas de rutas, Gateway Endpoints	1 min
2	`valheim-02-datos`	Bucket de mundos (versionado, lifecycle, eventos), bucket del panel, tabla DynamoDB con PITR; termination protection	1 min

#	STACK	QUÉ CREA	TIEMPO
3	`valheim-03-control`	6 Lambdas con sus logs, HTTP API con 6 rutas, 6 reglas de EventBridge, 2 DLQ, dashboard	2 min
4	`valheim-srv-<nombre>`	SG, NLB con 2 listeners UDP, Launch Template, ASG 0/1 con lifecycle hook, alarma, log group. Lo crea el panel o la consola	3 min

El stack de control recibe la clave de la API, el webhook de Discord, los límites y el cron de apagado; el de servidor recibe nombre, mundo, contraseña (`NoEcho`), tipo de instancia, minutos de inactividad y, si se quiere, una AMI propia. El UserData es el mismo para las dos AMI: con la base instala Docker y baja el juego (4 a 8 minutos medidos), con la propia solo restaura el mundo y arranca en un minuto. Del Learner Lab tuvimos en cuenta que no se pueden crear roles (usamos `LabRole` y `LabInstanceProfile`), que las instancias llegan hasta `large` y que las sesiones de 4 horas detienen las instancias al cerrar, así que la regla es cerrar con todo apagado. La guía paso a paso está en el `README.md` del paquete.

8. PRUEBAS

Probamos por capas y después de punta a punta en el Learner Lab, con el panel, la consola de AWS y el canal de Discord del grupo como testigos. Lo que verificamos:

- Infraestructura: los cuatro stacks desplegados de cero dos veces, con `CREATE_COMPLETE` al primer intento; drift detectado al tocar un tag a mano; termination protection que rechaza el borrado.
- Panel y API: clave incorrecta → 401; listado de servidores con su estado, dirección e historial; prender y apagar desde el botón.
- Ciclo del servidor: prender, instancia lanzada por el ASG, servidor En línea con la dirección del NLB en 4 minutos, conexión al juego, backup cada 20 minutos en S3, apagado por inactividad a los 30 minutos sin jugadores y apagado limpio con Run Command (eventos `shutdown` y `stopped`, mundo actualizado en S3).
- Crear y eliminar desde el panel: servidor nuevo listo para prender en 2 minutos, validaciones de nombre y contraseña (400) y nombre repetido (409), límite `MaxServers` (429), eliminación con el mundo guardado en S3.
- Avisos: cada paso llegó al Discord del grupo y quedó en el historial del panel.
- Pendiente: el contador de jugadores con alguien conectado, la persistencia del mundo entre apagado y prendido, la autorreparación tirando la instancia a mano, el 409 al eliminar un servidor prendido y la captura de Cost Explorer.

El bug que encontramos probando. Con `SERVER_PUBLIC=false` Valheim no respondía las consultas de estado y la métrica `PlayersOnline` publicaba 0 aunque hubiera gente; la alarma apagó el servidor a los 30 minutos con un jugador adentro. Lo arreglamos dejando el servidor público (con contraseña igual) y haciendo que `metric.sh` no publique nada si no puede leer el estado, así un fallo del contador nunca apaga una partida. Lo encontramos jugando, no leyendo el código.



Figura 6. Servidor clan-viernes creado desde el panel con el botón Nuevo servidor, ya en Apagado dos minutos después del pedido, con los eventos creating y stopped en el historial.

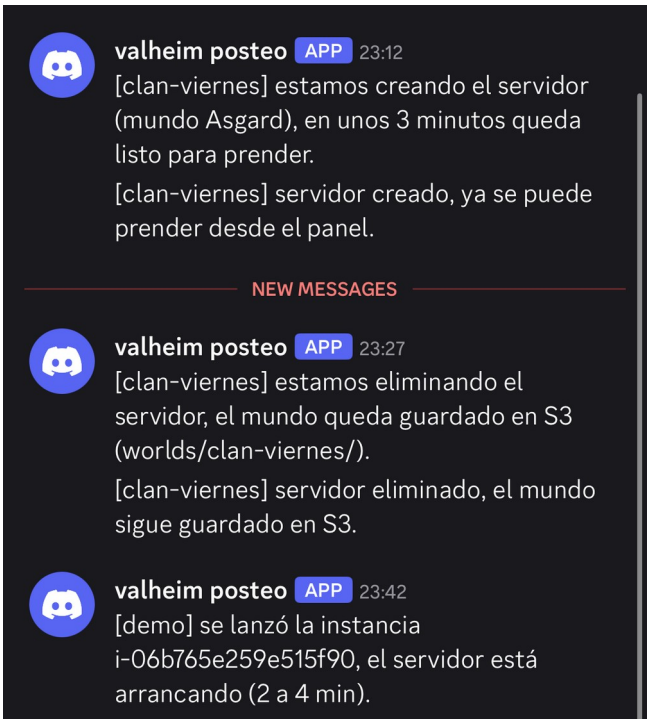


Figura 7. Avisos en el canal de Discord del grupo: creación y eliminación de clan-viernes y el arranque de demo.

9. LIMITACIONES Y PRÓXIMOS PASOS

LIMITACIÓN	POR QUÉ	PRÓXIMO PASO
No aguanta la caída de la región	Alcance y costo	Replicar el bucket de mundos y desplegar los templates en otra región
Clave compartida para el panel, sin usuarios	Simplicidad para el curso	Cognito con login y roles; clave en Parameter Store
Arranque de 4 a 8 minutos con la AMI base	Descarga de Steam en el primer arranque	AMI propia con el juego precargado (~1 minuto), ya prevista por parámetro
Hasta 20 minutos de juego perdidos si la instancia muere de golpe	Intervalo de backup	Bajar el intervalo o guardar en cada guardado del juego
Roles IAM amplios (`LabRole`)	Restricción del Learner Lab	Roles de mínimo privilegio por función en una cuenta propia
NLB caro para una sola instancia	Decisión para tener DNS fijo y health checks	Elastic IP cuando el costo mande; parámetro para elegir

Cosas que quedaron afuera por tiempo: restaurar una versión anterior del mundo desde el panel, horario de apertura automático, métricas de CPU en el panel y soportar otros juegos con el mismo plano de control.

10. CONCLUSIONES

Separar lo que administra (serverless) de lo que corre el juego (una instancia por servidor) fue lo que ordenó el resto: lo que administra cuesta centavos y escala solo, y lo único que cuesta plata de verdad está prendido solo mientras alguien juega. El ASG como interruptor, con el mundo en S3, convirtió la instancia en algo descartable y nos dio autorreparación sin hacer nada más. CloudFormation, además de desplegar, terminó siendo la API con la que el propio panel crea y elimina servidores.

Queda pendiente repetir la prueba del contador de jugadores con alguien conectado (la que falló la primera vez), la persistencia del mundo, la autorreparación tirando la instancia a mano, el 409 al eliminar un servidor prendido, armar la AMI propia y sacar la captura de Cost Explorer al cierre.